Traceveil Design Document

AI Assisted Smart Home IoT Digital Forensics and Incident Response Platform

Purpose

This five-page design review documents Traceveil as a local academic prototype for explainable investigation of smart-home IoT evidence. It is based on the repository contracts and implementation for evidence validation, canonical normalization, deterministic analysis, case APIs, dashboard presentation, and bounded local assistance.

Design conclusion

Traceveil is organized around one non-negotiable boundary: batch evidence and accepted laboratory telemetry become provenance-preserving canonical events before any finding is calculated. Python services are the analytical authority. The dashboard and optional local language model may present validated results, but neither may create, change, or recalculate forensic values.

Scope

In scope	Out of scope
Case and evidence management; dataset adapters; deterministic findings, risk, correlations, incidents, timelines and graph views; local report and assistant support.	Production-scale monitoring; third-party interception; unauthorized access; automatic containment; legal-admissibility claims; enterprise SIEM replacement.

Design goals

Goal	Design response
Evidence traceability	Hashes, validation receipts, source rows, raw-record hashes, adapter and normalization versions.
Repeatable triage	Stable filter order, versioned rules, bounded factorized risk and explicit correlation reasons.
Safe assistance	Evidence-grounded local narration with no authority over alerts, scores or evidence.
Investigator usability	Case-scoped REST resources and a React view for evidence, events, findings, timelines, graphs and reports.

Repository focus: the core forensic pipeline consists of evidence validation, canonical normalization, and deterministic analysis, with implemented case APIs, live telemetry ingestion, replayable real-time delivery, local evidence-grounded assistance, and reporting workflows. Physical laboratory acceptance and deployment-scale concerns remain outside the current validated scope.

1 Architecture and requirements

Traceveil uses two source paths that converge before analysis. Batch inputs include selected CASAS, TON IoT and controlled simulation profiles. The live path accepts authenticated and bounded laboratory telemetry through the implemented live-ingestion contract, with HTTP as the primary backend intake and optional MQTT bridging for controlled hardware integration. Both paths retain distinct source and ingestion provenance; the raw record remains authoritative for full review.

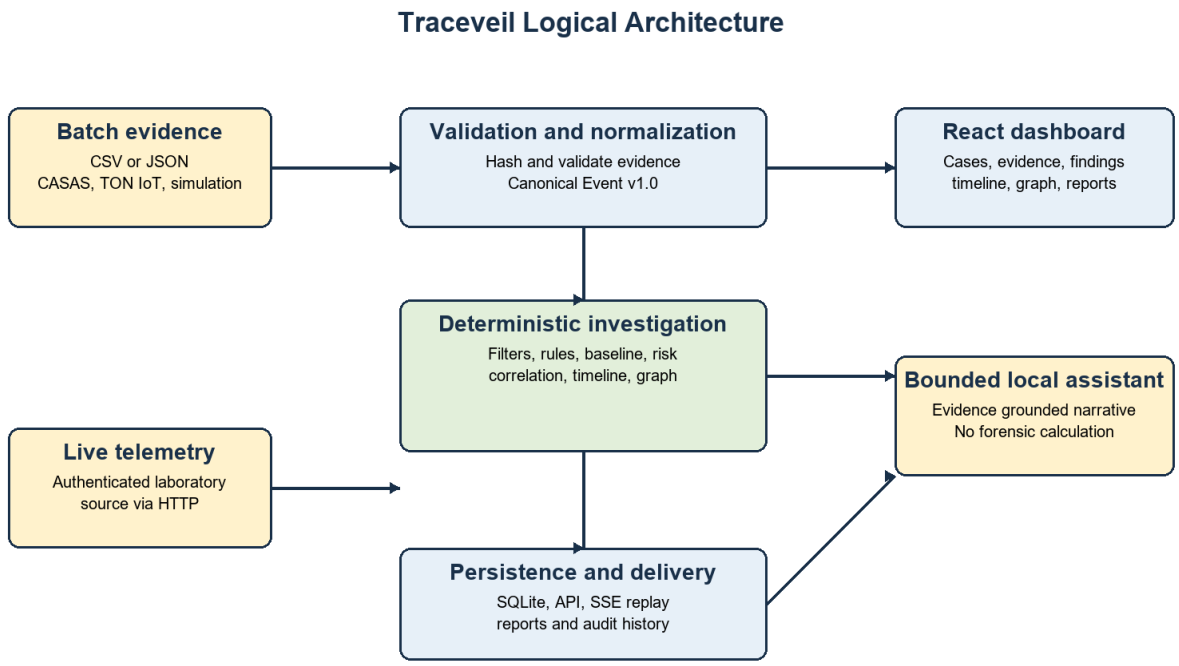

Figure 1. Logical architecture. Deterministic investigation is isolated from presentation and AI narration.

ID	Requirement	Implementation boundary
FR1	Validate CSV or JSON evidence and report row-level issues.	Evidence validation service; accepted, warning or rejected receipt.
FR2	Normalize batch and live records to one schema.	Canonical Event v1.0 and versioned adapters.
FR3	Calculate explainable investigation outputs.	AnalysisService rules, baselines, risk, correlation, incidents and aggregates.
FR4	Present results without client-side forensic calculation.	Case-scoped FastAPI routes and generated React DTOs.

Quality controls: input bounds; SHA-256 hashing; validation seals; caller-independent ingestion time; authenticated source tokens; rate limiting; deduplication; stable errors and request identifiers.

2 Data and analytical design

A canonical event records case and event identifiers, event type, optional observed time, backend-assigned ingestion time, entities, source label and attributes. Its provenance binds the event to evidence identity and hash, logical row or sequence, raw-record hash, origin, source identifiers, adapter version and normalization version. Observed and ingestion time never substitute for one another.

Deterministic pipeline

Order	Operation	Output
1	Validate case ownership, unique event IDs, filter and stable sort.	Selected ordered events.
2	Run versioned detection rules and behavioral baselines.	Findings with condition trace and evidence IDs.
3	Create explicit temporal/entity correlation edges.	Linked events and eligible incident components.
4	Calculate bounded five-factor risk and live-trigger alerts.	0-100 triage score and pending alerts.
5	Build stable timeline, graph and chart aggregates.	Backend-calculated investigation views.

Rule set and risk

Rule	Condition	Result
LABEL-001	Versioned selected source label or attack classification.	Medium, High, or Critical depending on normalized attack class; confidence 95.
RATE-001	requests >= three times positive baseline.	High, confidence 90.
AUTH-001	Ten failures for same known target and actor within 60 observed seconds.	Critical, confidence 98.
BASELINE-001	Metric exceeds mean + max(3 sigma, minimum delta).	High, confidence 80.

Risk weighting is severity 30%, confidence 25%, repetition 20%, device criticality 15%, and corroboration 10%. It is a deterministic triage signal, not proof of compromise, causation, attribution or intent. Correlation requires observed timestamps, a configured time window and a shared declared entity key; sharing a collector alone is insufficient.

Persistence and review

SQLite persists evidence, canonical events, analysis snapshots and independently queryable artifacts. Content-derived identifiers make an unchanged reanalysis idempotent; a reused identifier with different content is rejected. Batch findings remain findings, while alerts require a qualifying live triggering event.

3 UML behavior models

The use-case model identifies the investigator as the primary actor. A laboratory source is confined to telemetry submission, while the optional local assistant supports explanation only. The activity model makes validation and explicit investigator commit mandatory before normalization and analysis.

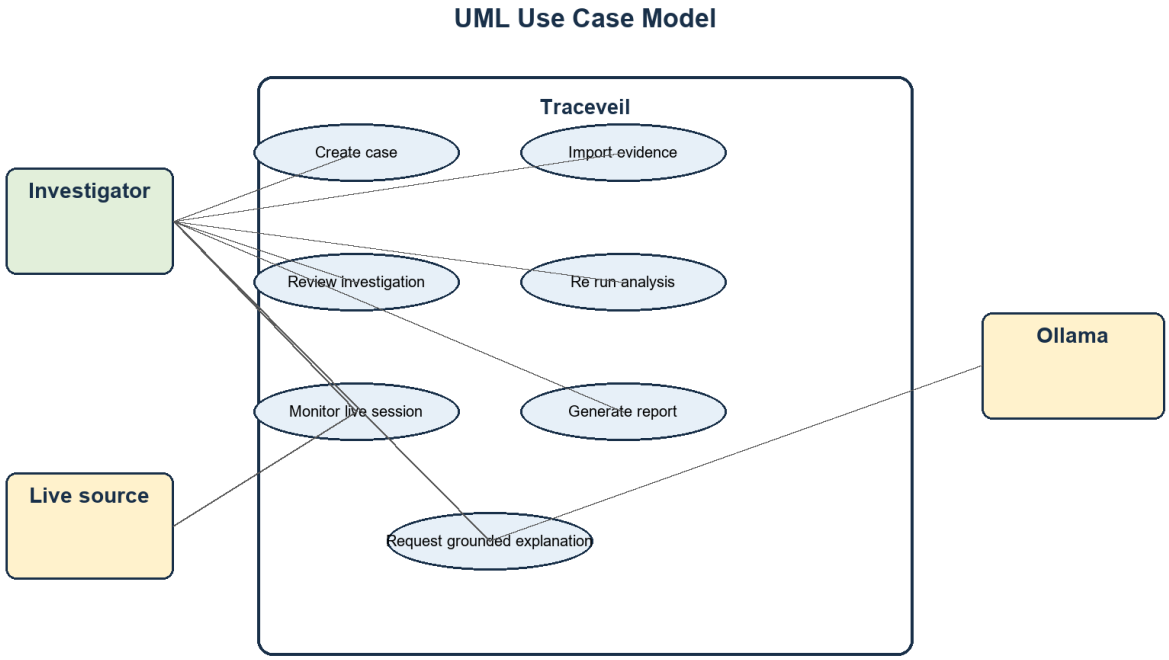

Figure 2. UML use-case model.

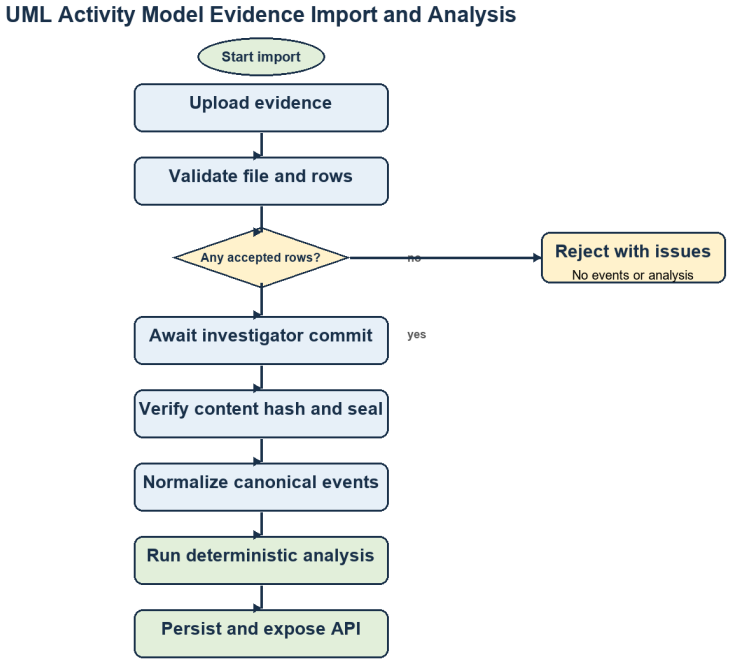

Figure 3. UML activity model for evidence import and deterministic analysis.

4 Live sequence, import state, and evaluation

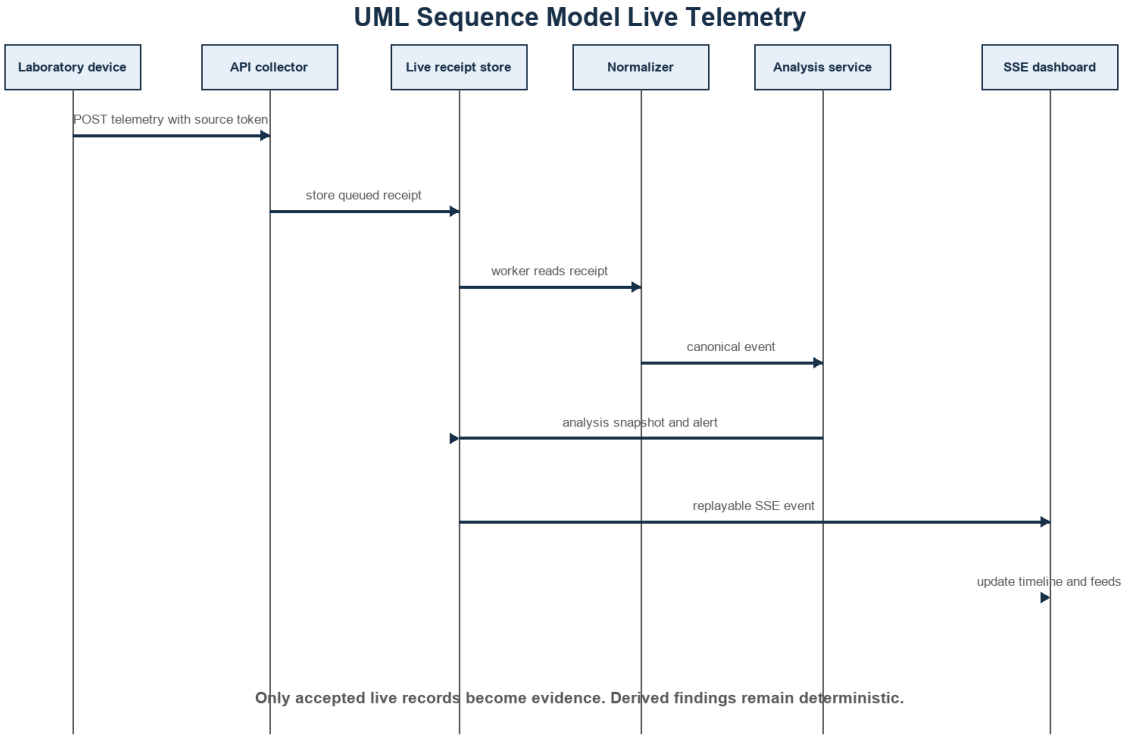

Figure 4. UML sequence model for accepted live telemetry and replayable SSE delivery.

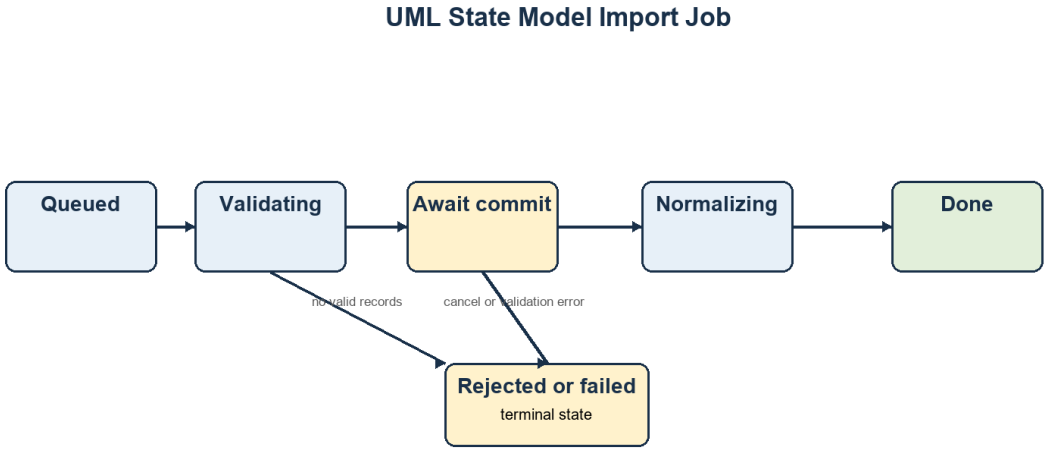

Figure 5. UML state model for the import job lifecycle.

Evaluation and limitations

Evaluate with controlled dataset fixtures and laboratory scenarios: validation acceptance and rejection, canonical mapping, repeated-run equality, rule traces, baseline samples, correlation reasons, timeline order, UI retrieval and live receipt behavior. The prototype is intentionally local and single-user; public dataset labels are evaluation signals rather than universal proof, correlations do not establish causation, and the local assistant is not an analytical authority. Software-level live ingestion, replay, deterministic analysis, and dashboard delivery are implemented; complete physical-device demonstration remains a controlled laboratory validation task.

Conclusion

Traceveil provides a coherent design for explainable IoT evidence investigation: validate first, preserve provenance, calculate deterministically, persist historical snapshots, and separate narrative assistance from forensic computation.